

Trees

What is a Tree

A tree is a hierarchical, non-linear data structure made of nodes connected by edges, with no cycles. It starts from a single 'root' node, and each node can have zero or more 'child' nodes. A node with no children is called a 'leaf'.

Key Terminology

- Root: the topmost node of the tree.
- Parent/Child: a node directly connected below another is its child.
- Leaf: a node with no children.
- Height: the number of edges on the longest path from a node to a leaf.
- Depth: the number of edges from the root to a given node.

Binary Tree

A binary tree is a tree where each node has at most two children, referred to as the left and right child. This is the most commonly used tree structure in CS coursework and interviews.

Binary Search Tree (BST)

A BST is a binary tree with an ordering rule: for every node, all values in its left subtree are smaller, and all values in its right subtree are larger. This ordering allows for efficient $O(\log n)$ search, insertion, and deletion — but only when the tree stays balanced. In the worst case (e.g. inserting sorted data), a BST degrades into a linked list with $O(n)$ operations.

Tree Traversal Methods

- Inorder (Left, Root, Right): visits nodes in ascending order for a BST.
- Preorder (Root, Left, Right): useful for copying/serializing a tree.
- Postorder (Left, Right, Root): useful for deleting a tree safely.
- Level-order (Breadth-First): visits nodes level by level using a queue.

BST Implementation in Python

```
class Node:

    def __init__(self, data):

        self.data = data

        self.left = None

        self.right = None
```

```

def insert(root, data):
    if root is None:
        return Node(data)
    if data < root.data:
        root.left = insert(root.left, data)
    else:
        root.right = insert(root.right, data)
    return root

```

```

def inorder(root, result):
    if root:
        inorder(root.left, result)
        result.append(root.data)
        inorder(root.right, result)
    return result

```

Balanced Trees

Self-balancing trees like AVL Trees and Red-Black Trees automatically restructure themselves during insertions/deletions to guarantee $O(\log n)$ operations, avoiding the worst-case degradation of a plain BST. These are heavily used internally in databases and language libraries (e.g. Java's TreeMap uses a Red-Black Tree).

Time Complexity (Balanced Tree)

- Search: $O(\log n)$.
- Insertion: $O(\log n)$.
- Deletion: $O(\log n)$.
- Traversal (any type): $O(n)$ — must visit every node.

Common Interview Problems

- Check if a binary tree is a valid BST.
- Find the lowest common ancestor (LCA) of two nodes.
- Determine if a tree is height-balanced.

- Serialize and deserialize a binary tree.
- Level-order traversal (BFS) and zigzag traversal.
- Find the diameter of a binary tree (longest path between any two nodes).